# Versioned Upgrade Design

## Purpose

This document explains the versioned-upgrade preparation that has been built into the current `PaperProof` contract system.

The goal is **not** to make the protocol hot-upgrade itself through ordinary governance parameters. Instead, the goal is to embed explicit upgrade handles so that future package upgrades can:

- reject stale state objects safely;
- migrate long-lived shared objects in a controlled way;
- keep core struct layouts stable; and
- support future security patches and protocol evolution without forcing an immediate v1-to-v2 state reset.

This design assumes the current architectural preference remains unchanged:

- core struct layouts should stay as stable as possible;
- most future changes should happen at the protocol behavior layer;
- future upgrades should be explicit, governed, and operationally controlled.

## Scope

The current versioned-upgrade preparation covers six long-lived shared objects:

1. `publishing::PaperRegistry`
2. `publishing::PaperRecord`
3. `comments::CommentsTree`
4. `governance::GovernanceVault`
5. `governance_voting::GovernanceConfig`
6. `governance_voting::Proposal`

These are the main state objects that define the long-lived protocol surface.

## Design Principles

The current design follows five principles.

## 1. Stable Core Layouts

The upgrade preparation is built around the assumption that the current core struct layouts are worth preserving.

That means future upgrades should prefer:

- new logic;
- stronger guards;
- new governance actions;
- new helper modules; and
- new integration modules

over repeated expansion or reshaping of the current core state structs.

## 2. Versioned Shared Objects

Every covered shared object now carries its own `version` field.

This allows future packages to distinguish between:

- objects already migrated to the current logic generation; and
- objects that still belong to an older generation.

## 3. Version Guards on Critical Entrypoints

Critical public protocol entrypoints now assert that the relevant shared object is at the current supported version.

This means future package upgrades can intentionally invalidate stale objects until a migration path is run.

## 4. Controlled Migration Hooks

Each major object family now exposes a `migrate_*` entrypoint.

These migration hooks are the future execution points for:

- object version bumps;
- migration-time invariant repair;

- logic-specific state normalization; and
- future one-time transition steps.

### 5. Upgrade Authority Alignment

The migration hooks are not left open to arbitrary callers.

They are controlled through the protocol's recorded `upgrade_authority`, which keeps future package-upgrade operations and future state-migration operations aligned under one recognized control path.

## Current Object Versions

The current implementation uses the following version constants:

- `governance::GOVERNANCE_VAULT_VERSION = 1`
- `governance_voting::GOVERNANCE_CONFIG_VERSION = 1`
- `governance_voting::PROPOSAL_VERSION = 1`
- `publishing::PUBLISHING_REGISTRY_VERSION = 1`
- `publishing::PUBLISHING_RECORD_VERSION = 1`
- `comments::COMMENTS_TREE_VERSION = 1`

At present, all newly created objects start at version `1`.

## Current Version Guards

### Governance Layer

The governance layer now provides:

- `assert_current_vault`
- `assert_upgrade_authority`

These are used to guard:

- fee setters;
- fee collection helpers;
- operator nomination / acceptance / cancellation;

- governance-voting execution;
- migration entrypoints.

## Governance Voting Layer

The voting layer now provides:

- `assert_current_config`
- `assert_current_proposal`

These are used to guard:

- proposal creation;
- vote casting;
- proposal finalization;
- proposal execution;
- token claim;
- migration entrypoints.

## Publishing Layer

The publishing layer now provides:

- `assert_current_registry`
- `assert_current_record`

These are used to guard:

- `reserve_code`
- `finalize_paper`
- `add_version`
- `transfer_paper_owner`
- `record_storage_extension`
- `extend_walrus_storage_and_record`
- `set_paused`
- `update_limits`
- `set_ui_status`

### Comments Layer

The comments layer now provides:

- `assert_current_tree`

This is used to guard:

- `add_onchain_comment`
- `add_blob_comment`
- `like_paper`
- `unlike_paper`
- `set_tree_status`
- `set_comment_status`
- `transfer_tree_owner`

## Current Migration Hooks

The following migration hooks now exist.

### Governance

- `governance::migrate_vault`

### Governance Voting

- `governance_voting::migrate_config`
- `governance_voting::migrate_proposal`

### Publishing

- `publishing::migrate_registry`
- `publishing::migrate_record`

### Comments

- `comments::migrate_tree`

At the current stage, these hooks are intentionally minimal. They currently:

- verify the relevant authority path;
- verify registry alignment where relevant;
- verify that the old version is not ahead of the package-supported version;
- update the object's version to the current version when needed.

They are designed to become the canonical place for future migration logic.

## Upgrade Authority Model

The migration path is intentionally tied to the recorded `upgrade_authority`.

This means:

- ordinary users cannot trigger migrations;
- ordinary operators cannot trigger migrations unless they are also the protocol-recognized upgrade authority;
- future state transitions can be coordinated under the same authority that is supposed to control package upgrades.

Important boundary:

- the protocol can record and govern the official `upgrade_authority`;
- actual Sui package upgrades still depend on the real `UpgradeCap` being held by that address or custody path.

So the contract-level design and the actual Sui package-upgrade custody must be kept operationally aligned.

## Why This Is Useful

This design is especially useful for the kinds of future upgrades most likely to matter for `PaperProof`.

### Security Patch Upgrades

The current protocol may later need behavior-layer fixes in areas such as:

- governance voting;
- permission checks;

- fee enforcement;
- comments behavior rules;
- executable governance action routing.

Version guards make it possible to ensure that upgraded logic only runs against state that has been explicitly migrated or confirmed current.

### Business Evolution Upgrades

The current protocol may later add:

- treasury integration;
- incentive or points logic;
- additional governance actions;
- stronger moderation policy layers;
- richer execution or upgrade workflows.

Migration hooks provide a place to introduce any one-time state transition that those features may require, while still keeping the core object families stable.

## What This Design Does Not Mean

This design does **not** mean:

- the current protocol is self-upgrading on-chain;
- governance parameters alone can rewrite contract code;
- future upgrades can arbitrarily reshape core struct layouts without planning;
- Sui package compatibility constraints disappear.

Instead, it means the protocol is now prepared for a more disciplined future upgrade process.

## How Future Upgrades Should Be Done

The expected future workflow is:

### Step 1. Prepare a New Package Version

Build the next package version with:

- new behavior logic;
- updated version constants if needed;
- any additional migration logic inside the existing `migrate_*` hooks.

At this stage, the new package should define:

- what object versions it accepts;
- what object versions it upgrades from;
- what state normalization or one-time rewrite steps are required.

## Step 2. Publish the Package Upgrade Through Sui's Upgrade Path

Upgrade the package using the real `UpgradeCap`.

Operationally, this should be done by the party or custody path that matches the protocol's recorded `upgrade_authority`.

## Step 3. Run Object Migrations

After the new package is live, call the appropriate migration hooks:

- `migrate_vault`
- `migrate_config`
- `migrate_proposal`
- `migrate_registry`
- `migrate_record`
- `migrate_tree`

depending on which object classes need to move to the new version.

At this stage, the package can:

- bump object versions;
- rewrite fields if a compatible migration path exists;
- repair invariants;
- populate new derived state;
- normalize legacy state to new assumptions.

### Step 4. Operate Only on Current Objects

Once migration is complete, the new package should operate on:

- current-version objects only

through the existing version guards.

If an old object was not migrated, entrypoint guards should continue to reject it until it is migrated.

### Step 5. Update Frontend and Operational Routing

After package upgrade and migration:

- update the official frontend
- update operational scripts
- update any indexer or monitoring logic
- update documentation where needed

This step is especially important when new behavior or new governance actions have been added.

## Practical Upgrade Strategy for PaperProof

Given the current protocol direction, the most likely future candidates for versioned-upgrade use are:

- governance voting rules
- permission / authority checks
- fee enforcement logic
- comments behavior rules
- governance action expansion
- treasury integration
- future incentive logic

The current versioned-upgrade preparation is intentionally aimed at those behavior-layer evolutions rather than aggressive core state redesign.

## Test Coverage

The current test suite now includes dedicated checks for the upgrade hooks:

- `governance_tests::test_vault_defaults_and_fee_setters`
- `governance_voting_tests::test_migrate_config_and_proposal_hooks`
- `publishing_tests::test_registry_and_record_migration_hooks`
- `comments_tests::test_tree_migration_hook`

These tests currently verify that:

- the exposed version getters match the current constants;
- the migration hooks are callable by the correct authority path; and
- the upgrade handles do not break the existing business logic.

## Summary

The current `PaperProof` contracts now include explicit versioned–upgrade handles without changing the architectural preference for stable core structs.

The key outcome is:

- core shared objects now carry versions;
- critical entrypoints now reject unsupported versions;
- controlled migration hooks now exist;
- upgrade control is aligned with the protocol's recognized upgrade authority;
- and future package upgrades now have a clean place to attach safe migration logic.

This makes the protocol materially better prepared for both:

- future security–patch upgrades; and
- future business or governance evolution.